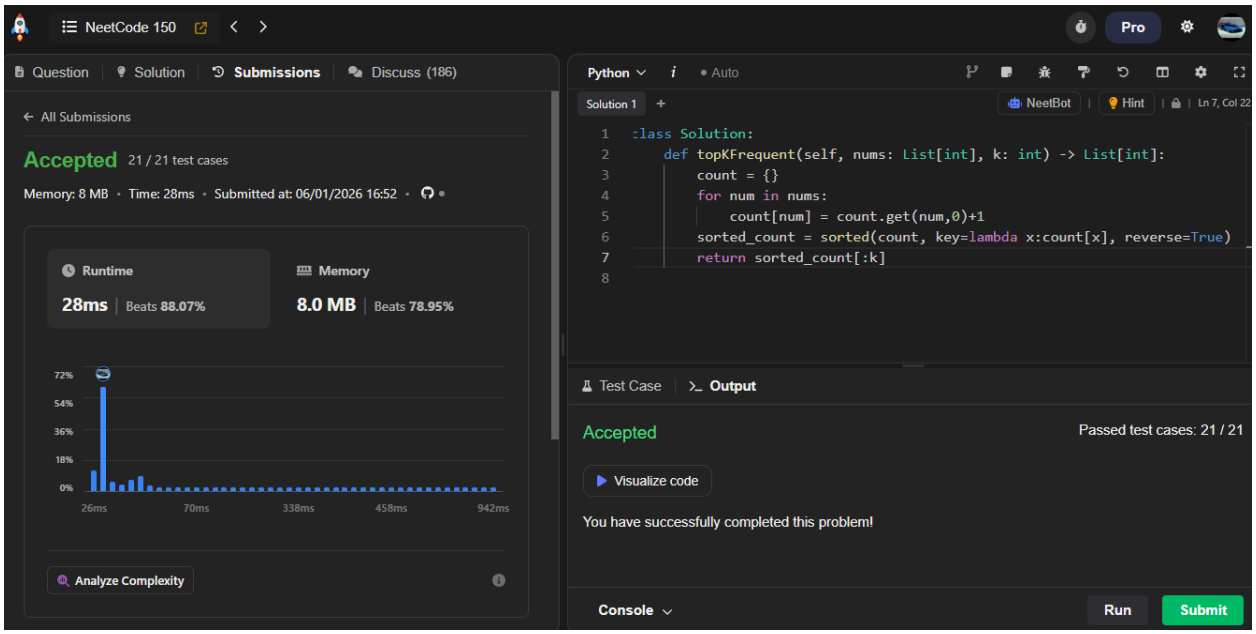
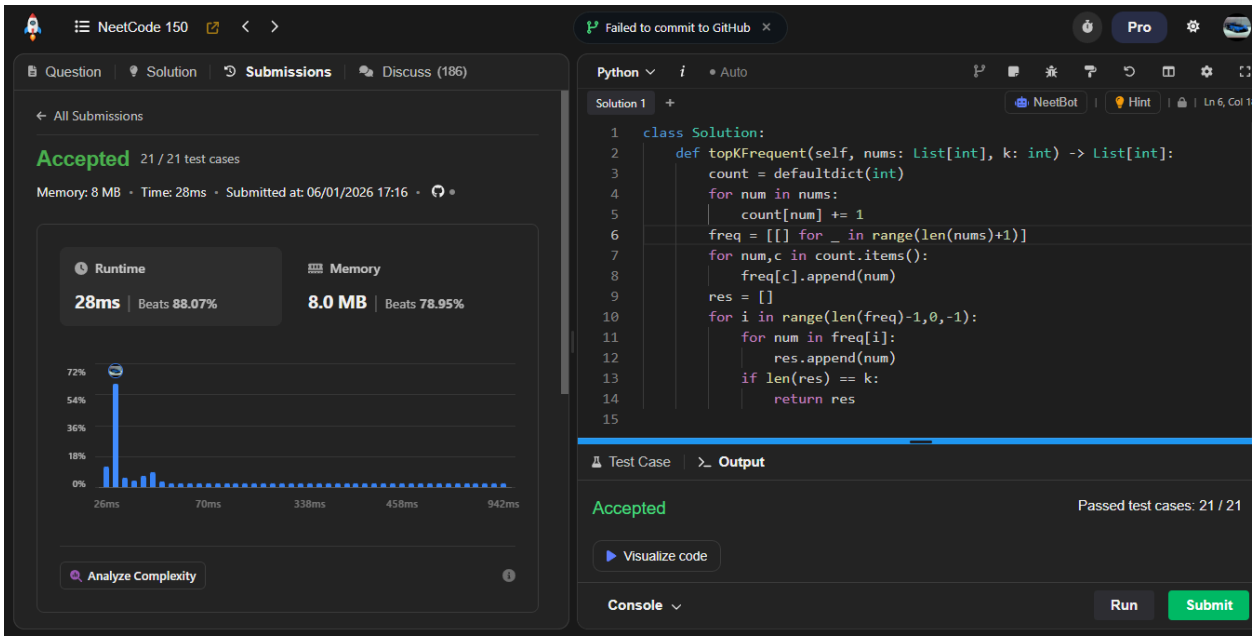
## *Top K frequent elements*

1. Brute force method




2. Optimal approach → hash map  bucket sort



***Comparison:***

### General Overview

| | Brute Force | Optimized |
|---|---|---|
| Approach | Count + Sort | Count + Bucket Index |
| Time Complexity | O(n log n) | O(n) |
| Space Complexity | O(n) | O(n) |
| Lines of Code | Less | More |
| Difficulty to Code | Easy | Medium |
| Chance of Bugs | Low | Medium |
| Best for Large Input | ❌ | ✅ |

### Step by Step

| Step | Brute Force | Optimized |
|---|---|---|
| Step 1 | Count freq using HashMap | Count freq using HashMap |
| Step 2 | Sort keys by frequency | Create bucket array size n+1 |
| Step 3 | Slice first k elements | Read right to left, collect k |
| Bottleneck | Sorting → O(n log n) | No bottleneck → O(n) |

### Key Concepts Used

| | Brute Force | Optimized |
|---|---|---|
| Data Structure | HashMap | HashMap + Array |
| Algorithm | Sorting | Bucket Sort |
| Key Insight | Sort by value of HashMap | Frequency as array index |
| Built-in Used | sorted() | None |